

计算物理学作业

1900011604 张植竣 北京大学

2021 年 10 月 16 日

1.(a) 由题意，估计最大可能上限，则假设每一项的舍入误差均为 $\frac{\epsilon_M}{2}$ 。

总和的精确值为 $\sum_{i=1}^N x_i = N\bar{x}$ ，相比于 $\bar{x}$ ，其位数高了 $\log_2 N$ ，这会导致 $N \cdot \frac{N\epsilon_M}{2} = \frac{N^2\epsilon_M}{2}$ 的舍入误差，而最后除以 N 再取平均值又会引起 $\frac{\epsilon_M}{2}$ 的舍入误差，故最大的误差值估计为 $\frac{(N+1)\epsilon_M}{2}$ 。

1.(b) 第二个公式更为稳定和准确，因为第二个公式是较小的数相减再平方相加，而第一个公式是计算平方后再让两个大数相减，在平方项求和及大数相减过程中会丢失很多高位信息。

1.(c) 证明如下：

$$I_0 = \int_0^1 \frac{1}{x+5} dx = \ln(x+5)|_0^1 = \ln \frac{6}{5}$$

而且：

$$\begin{aligned} I_k + 5I_{k-1} &= \int_0^1 \frac{1}{x+5} dx + \int_0^1 \frac{1}{x+5} dx \\ &= \int_0^1 \frac{x^k + 5x^{k-1}}{x+5} dx \\ &= \int_0^1 \frac{(x+5)x^{k-1}}{x+5} dx \\ &= \int_0^1 x^{k-1} dx \\ &= \frac{x^k}{k} \Big|_0^1 \\ &= \frac{1}{k} \end{aligned}$$

如果在计算 I_0 时有一个微小的误差 ϵ ，利用上面的递推公式时：

$$I_n = \frac{1}{n} - 5I_{n-1} = \frac{1}{n} - 5\left(\frac{1}{n} - 5I_{n-2}\right) = \dots$$

可见最终误差应为 $\epsilon_n = 5^n \epsilon$ ，以指数形式增长，说明利用该递推公式计算 I_n 是不稳定的。

2.(a) 上三角矩阵的行列式是对角线元素的乘积： $\det A = 1$

2.(b) 用初等行变换法可以发现：为了将 A 化为单位矩阵，需要先将第 $n-1$ 行与第 n 行相加，再将第 $n-2$ 行与更改后的第 $n-1$ 行相加……体现在单位矩阵 I 的变化上就是：

$$A_{ij}^{-1} = \sum_{x=i+1}^n A_{xj}^{-1} = 1 + \sum_{x=1}^{j-i-2} 2^{x-1} = 2^{j-i-1} \quad (i > j)$$

故 A^{-1} 的形式为:

$$A_{ij}^{-1} = \begin{cases} 0, & i < j \\ 1, & i = j \\ 2^{i-j-1}, & i > j \end{cases}$$

2.(c) 首先证明: $\frac{\|A\mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty}$ 在 $\mathbf{x}$ 的各分量相等时取最大值。

设 $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$, 则它也可以被看作是一个 $n \times 1$ 的矩阵 $\mathbf{x}$, 且 $x_i = x_{i1}$:

$$\frac{\|A\mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty} = \frac{\max_i \left| \sum_{j=1}^n A_{ij} x_{j1} \right|}{\max_i |x_{i1}|}$$

假如 $\mathbf{x}$ 中除 x_k 外, 其余所有元素均相等且等于 x_0 , 则分两种情况:

(1) $|x_k| > |x_0|$, 则 $\left| \sum_{j=1}^n A_{ij} x_{j1} \right| \leq \left| \sum_{j=1}^n A_{ij} x_k \right|$, 即:

$$\frac{\|A\mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty} = \frac{\max_i \left| \sum_{j=1}^n A_{ij} x_{j1} \right|}{\max_i |x_{i1}|} = \frac{\max_i \left| \sum_{j=1}^n A_{ij} x_{j1} \right|}{|x_k|} \leq \frac{\max_i \left| \sum_{j=1}^n A_{ij} x_k \right|}{|x_k|}$$

(2) $|x_k| < |x_0|$, 则 $\left| \sum_{j=1}^n A_{ij} x_{j1} \right| \leq \left| \sum_{j=1}^n A_{ij} x_0 \right|$, 即:

$$\frac{\|A\mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty} = \frac{\max_i \left| \sum_{j=1}^n A_{ij} x_{j1} \right|}{\max_i |x_{i1}|} = \frac{\max_i \left| \sum_{j=1}^n A_{ij} x_{j1} \right|}{|x_0|} \leq \frac{\max_i \left| \sum_{j=1}^n A_{ij} x_0 \right|}{|x_0|}$$

故 $\frac{\|A\mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty}$ 在 $\mathbf{x}$ 的各分量相等时取最大值。设 $\mathbf{x}$ 的各分量均为 x_0 , 则:

$$\frac{\|A\mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty} = \frac{\max_i \left| \sum_{j=1}^n A_{ij} x_{j1} \right|}{\max_i |x_{i1}|} = \frac{\max_i \left| \sum_{j=1}^n A_{ij} x_0 \right|}{|x_0|} = \frac{\max_i \sum_{j=1}^n |A_{ij}| |x_0|}{|x_0|} = \max_i \sum_{j=1}^n |A_{ij}|$$

则 ∞ 模为:

$$\|A\|_\infty = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty} = \max_i \sum_{j=1}^n |A_{ij}|$$

即提取出矩阵 A 各元素和的绝对值最大的一行。

2.(d) 依次证明如下:

(1) 证明对于一个幺正矩阵 $U \in \mathbb{C}^{n \times n}$, $\|U\|_2 = \|U^\dagger\|_2 = 1$:

幺正矩阵的性质为 $U^\dagger U = U U^\dagger = I$, 而单位矩阵 I 的特征值均为 1, 则其谱半径 $\rho(I) = \max \lambda(I) = 1$ 。

又因为对于任意方阵 B , 其 Euclid 模 $\|B\|_2 = \sqrt{\rho(B^\dagger B)}$, 则 $\|U\|_2 = \sqrt{\rho(U^\dagger U)} = \sqrt{\rho(I)} = 1$ 。

(2) 证明对于任意一个复方阵 $A \in \mathbb{C}^{n \times n}$, $\|UA\|_2 = \|A\|_2$:

由于 $U^\dagger U = I$, $(UA)^\dagger = A^\dagger U^\dagger$, 则 $\|UA\|_2 = \sqrt{\rho(A^\dagger U^\dagger U A)} = \sqrt{\rho(A^\dagger A)} = \|A\|_2$;

同理，对于 $(UA)^{-1} = A^{-1}U^{-1}$ ，有 $(A^{-1}U^{-1})^\dagger = (A^{-1}U^\dagger)^\dagger = (A^{-1})^\dagger U$ ，易得 $\|(UA)^{-1}\|_2 = \|A^{-1}\|_2$ ；

故 $K_2(A) = \|A\|_2 \|A^{-1}\|_2 = \|UA\|_2 \|(UA)^{-1}\|_2 = K_2(UA)$ 。

2.(e) 对于一个所有对角元都为 1，而所有的上三角部分矩阵元都是 -1 的矩阵 A ，其 ∞ 模为：

$$\|A\|_\infty = \max_i \sum_{j=1}^n |A_{ij}| = 1$$

其逆矩阵 A^{-1} 满足：

$$A_{ij}^{-1} = \begin{cases} 0, & i < j \\ 1, & i = j \\ 2^{j-i-1}, & i > j \end{cases}$$

显然 A^{-1} 各元素和的绝对值最大的一行就是第一行，则其 ∞ 模为：

$$\|A^{-1}\|_\infty = \max_i \sum_{j=1}^n |A_{ij}| = 1 + \sum_{x=2}^n 2^{x-2} = 2^{n-1}$$

故 A 的条件数为：

$$K_\infty(A) = \|A\|_\infty \|A^{-1}\|_\infty = 2^{n-1}$$

3.(a) 将积分 D 对 c_i 求偏导，令其为 0：

$$\begin{aligned} \frac{\partial D}{\partial c_i} &= \frac{\partial}{\partial c_i} \int_0^1 \left(\sum_{i=1}^n c_i x^{i-1} - f(x) \right)^2 dx \\ &= \int_0^1 \frac{\partial}{\partial c_i} \left(\sum_{j=1}^n c_j x^{j-1} - f(x) \right)^2 dx \\ &= \int_0^1 2 \left(\sum_{j=1}^n c_j x^{j-1} - f(x) \right) x^{i-1} dx \\ &= 0 \end{aligned}$$

显然这等价于：

$$\int_0^1 x^{i-1} \sum_{j=1}^n c_j x^{j-1} dx = \int_0^1 x^{i-1} f(x) dx$$

由于积分与求和可交换，有：

$$\begin{aligned} \sum_{j=1}^n \int_0^1 x^{i-1} c_j x^{j-1} dx &= \sum_{j=1}^n c_j \int_0^1 x^{i+j-2} dx \\ &= \sum_{j=1}^n c_j \cdot \left(\frac{1}{i+j-1} \right) dx \\ &= \int_0^1 x^{i-1} f(x) dx \end{aligned}$$

故 Hilbert 矩阵 $H = H_n$ 的矩阵元为：

$$H_{ij} = \frac{1}{i+j-1}$$

矢量 $\mathbf{b}$ 的各分量为：

$$b_i = \int_0^1 x^{i-1} f(x) dx$$

3.(b) 首先, 因为 $H_{ij} = H_{ji} = \frac{1}{i+j-1}$, 所以 H 为对称矩阵。

又因为 $\mathbf{c}^T \mathbf{H} \mathbf{c} = \mathbf{c}^T \mathbf{b}$, 且有:

$$\begin{aligned} 2\mathbf{c}^T \mathbf{b} &= 2 \sum_{i=1}^n c_i \int_0^1 f(x) x^{i-1} dx \\ &= 2 \int_0^1 f(x) \sum_{i=1}^n c_i x^{i-1} dx \\ &= \int_0^1 \left[\left(\sum_{i=1}^n c_i x^{i-1} \right)^2 + f^2(x) \right] dx - \int_0^1 \left[\sum_{i=1}^n c_i x^{i-1} - f(x) \right]^2 dx \end{aligned}$$

取 $f(x) = \sum_{i=1}^n c_i x^{i-1}$, 代入得:

$$\mathbf{c}^T \mathbf{b} = \int_0^1 \left(\sum_{i=1}^n c_i x^{i-1} \right)^2 dx \geq 0$$

等号只有在 c_i 都等于 0 时才取得, 故 H 是正定的。

而正定矩阵的行列式大于 0, 则 $\det H > 0$, 故 H 也是非奇异的。

3.(c) 对 $\det H_n$ 取自然对数:

$$\ln \det H_n = \ln \frac{c_n^4}{c_{2n}} = \ln c_n^4 - \ln c_{2n}$$

又因为 $c_n = \prod_{i=1}^{n-1} i!$, 则 $\ln c_n = \sum_{i=1}^{n-1} \ln i!$ 。

代入 Sterling 公式 $\ln n! \approx n \ln n - n + \frac{1}{2} \ln(2\pi n)$, 并用积分近似求和, 得到:

$$\begin{aligned} \ln c_n &\approx \sum_{i=1}^{n-1} \left[i \ln i - i + \frac{1}{2} \ln(2\pi i) \right] \\ &= \sum_{i=1}^{n-1} i \ln i - \frac{n(n-1)}{2} + \frac{n-1}{2} \ln(2\pi) + \frac{1}{2} \sum_{i=1}^{n-1} \ln i \\ &\approx \int_1^{n-1} x \ln x dx - \frac{n(n-1)}{2} + \frac{n-1}{2} \ln(2\pi) + \frac{1}{2} \int_1^{n-1} \ln x dx \\ &= \frac{1}{4} x^2 (2 \ln x - 1) \Big|_1^{n-1} - \frac{n(n-1)}{2} + \frac{n-1}{2} \ln(2\pi) + \frac{1}{2} (x \ln x - x) \Big|_1^{n-1} \\ &= \left[\frac{(n-1)^2}{2} \ln(n-1) - \frac{(n-1)^2}{4} + \frac{1}{4} \right] - \frac{n(n-1)}{2} + \frac{n-1}{2} \ln(2\pi) + \frac{1}{2} [(n-1) \ln(n-1) - n] \\ &= \frac{n^2 - n}{2} \ln(n-1) - \frac{3n^2 - 2n}{4} + \frac{n-1}{2} \ln(2\pi) \end{aligned}$$

则有:

$$\begin{aligned} \ln c_n^4 &= 4 \ln c_n = (2n^2 - 2n) \ln(n-1) - (3n^2 - 2n) + (2n-2) \ln(2\pi) \\ \ln c_{2n} &= (2n^2 - n) \ln(2n-1) - (3n^2 - n) + \frac{2n-1}{2} \ln(2\pi) \end{aligned}$$

于是得到:

$$\ln \det H_n = \ln c_n^4 - \ln c_{2n} \sim -2n^2 \ln 2 + n \ln(2\pi)$$

故 $\det H_n$ 近似为:

$$\det H_n \sim 4^{-n^2} (2\pi)^n$$

3.(d) 高斯消元法求解任意线性方程组的代码如下：

```
1  import numpy as np
2
3
4  def is_zero(a): # 判定一个向量 a 是否为零向量，精确到 10^{-15}
5      k = 0
6      for i in range(len(a)):
7          if abs(a[i]) < 10e-15:
8              k += 1
9      if k == len(a):
10         return True
11     else:
12         return False
13
14
15 def Gauss(A, b): # 对增广矩阵使用 Gauss 消元法
16     n = len(A) # 系数矩阵的行数
17     m = len(A[0]) # 系数矩阵的列数（未知量的个数）
18     A = np.array(A, dtype=float)
19     b = np.array(b, dtype=float)
20     Ab = np.column_stack((A, b)) # 生成一个增广矩阵 (A, b)
21     # 部分支点遴选
22     for j in range(0, m - 1):
23         pivot_list = []
24         for r in range(j, n):
25             pivot_list.append(abs(Ab[r, j]))
26         # 从第 j 列的第 j 行到第 n 行之间选出绝对值最大的矩阵元
27         Max = max(pivot_list)
28         r = pivot_list.index(Max) + j
29         # 交换第 j 列和第 r 列，此时第 j 列第 j 行的对角元成为该列的最大元
30         if r != j:
31             Ab[[j, r], :] = Ab[[r, j], :]
32         # 利用支点元将该列第 j+1 行到第 n 行的元素全部消为 0
33         for i in range(j + 1, n):
34             Ab[i] = Ab[i] - Ab[j] * Ab[i, j] / Ab[j, j]
35     # 删除增广矩阵的零行
36     i = 0
37     n = len(Ab)
38     while i < n:
39         if is_zero(Ab[i]) == True:
40             Ab = np.delete(Ab, i, axis=0)
41             n -= 1
42             i += 1
43     n = len(Ab)
44     m = len(Ab[0]) - 1
45     # 判定方程组解的情况
```

```

46     if n < m: # 非零行个数
47         return "Infinity or None" # 无穷多解或零解
48     elif n > m: # 系数矩阵的行秩小于行数，还有可以消去的零行
49         for i in range(n - 1, m - 1, -1):
50             Ab[i] = Ab[i] - Ab[i - 1] * Ab[i, m - 1] / Ab[i - 1, m - 1]
51             if is_zero(Ab[i]) is True:
52                 Ab = np.delete(Ab, i, axis=0)
53     n = len(Ab)
54     m = len(Ab[0]) - 1
55     # print(Ab, "\n")
56     # 反代法
57     x = np.zeros(n)
58     # 首先解出  $x_{\{n\}}$ 
59     if Ab[n - 1][m - 1] != 0:
60         x[n - 1] = Ab[n - 1][m] / Ab[n - 1][m - 1]
61     else:
62         return "None" # 零解
63     # 之后解出  $x_{\{1\}}$  到  $x_{\{n-1\}}$ 
64     for i in range(n - 2, -1, -1):
65         Sum = 0
66         for k in range(i, m):
67             Sum += Ab[i][k] * x[k]
68         x[i] = (Ab[i][m] - Sum) / Ab[i][i]
69     return x
70
71
72 def H_matrix_func(n): # 生成一个 Hilbert 矩阵 A
73     A = np.zeros((n, n))
74     for i in range(1, n + 1):
75         for j in range(1, n + 1):
76             A[i - 1, j - 1] = 1 / (i + j - 1)
77     return A
78
79
80 def question(n): # 求解线性方程  $Hx = b$ 
81     H = H_matrix_func(n)
82     b = [1 for i in range(n)]
83     x = Gauss(H, b)
84     return x
85
86
87 for i in range(1, 11):
88     x = question(i)
89     print(list(x))

```

$n = 1$ 到 $n = 10$ 的各输出结果如下：

```

1      [1.0]
2      [-2.0000000000000001, 6.000000000000002]
3      [3.00000000000000195, -24.000000000000092, 30.00000000000008]
4      [-3.9999999999993143, 59.9999999999208, -179.9999999999808, 139.99999999998747]
5      [4.999999999999574, -120.00000000002163, 630.0000000001703, -1120.000000000335,
        630.0000000001911]
6      [-6.000000001276931, 210.00000003757168, -1680.000000258774, 5040.000000681161,
        -6300.0000007583185, 2772.000000300736]
7      [7.000000040402028, -336.00000160737403, 3780.000015434454, -16800.000059826783,
        34650.00010939741, -33264.00009431933, 12012.000030908977]
8      [-7.9999995398738974, 503.9999724640256, -7559.999612741079, 46199.99778858559,
        -138599.99379808275, 216215.99093752247, -168167.99338049823,
        51479.99809169218]
9      [8.999921460705082, -719.9944911930725, 13859.905611358774, -110879.31972240417,
        450447.48562329676, -1009002.8337807284, 1261254.03568819,
        -823676.3814356452, 218789.10252371323]
10     [-9.99704968383594, 989.7445331886411, -23754.54997029394, 240190.39320730607,
        -1261023.1592328737, 3783128.4319968503, -6725650.321446537,
        7000245.761976883, -3937676.808608159, 923660.5025033345]

```

因为 Hilbert 矩阵是实对称矩阵，则它一定是 Hermite 矩阵, 可以做 Cholesky 分解，代码如下：

```

1  import numpy as np
2
3
4  def Cholesky(A, b): # 对原矩阵使用 Cholesky 分解
5      n = len(A) # 原矩阵的阶数
6      A = np.array(A, dtype=float)
7      b = np.array(b, dtype=float)
8      # 设  $A = h_{\text{dagger}} h$ ，其中  $h$  为下三角矩阵
9      h = np.zeros((n, n))
10     h[0, 0] = A[0, 0] ** 0.5 # 首先令  $h_{11} = \sqrt{A_{11}}$ 
11     for i in range(1, n):
12         # 计算  $h_{ij}$ 
13         for j in range(i):
14             Sum1 = 0
15             for k in range(j):
16                 Sum1 += h[i, k] * h[j, k]
17             h[i, j] = (A[i, j] - Sum1) / h[j, j]
18         # 计算  $h_{ii}$ 
19         Sum2 = 0
20         for k in range(i):
21             Sum2 += h[i, k] ** 2
22         h[i, i] = (A[i, i] - Sum2) ** 0.5
23     h_dagger = np.zeros((n, n))
24     for i in range(n):

```

```

25         for j in range(n):
26             h_dagger[i, j] = h[j, i].conjugate()
27     # 对方程进行反代求解:  $h y = b$ ,  $h\_dagger x = y$ 
28     #  $B = h y = b$ 
29     B = np.column_stack((h, b))
30     y = np.zeros(n)
31     # 因为  $h$  是下三角矩阵, 这里的反代顺序与通常的相反
32     y[0] = B[0, n] / B[0, 0]
33     for i in range(1, n):
34         Sum3 = 0
35         for k in range(0, i):
36             Sum3 += B[i][k] * y[k]
37         y[i] = (B[i][n] - Sum3) / B[i][i]
38     #  $C = h\_dagger x = y$ 
39     C = np.column_stack((h_dagger, y))
40     x = np.zeros(n)
41     x[n - 1] = C[n - 1][n] / C[n - 1][n - 1]
42     for i in range(n - 2, -1, -1):
43         Sum4 = 0
44         for k in range(i, n):
45             Sum4 += C[i][k] * x[k]
46         x[i] = (C[i][n] - Sum4) / C[i][i]
47     return x
48
49
50 def H_matrix_func(n): # 生成一个 Hilbert 矩阵 A
51     A = np.zeros((n, n))
52     for i in range(1, n + 1):
53         for j in range(1, n + 1):
54             A[i - 1, j - 1] = 1 / (i + j - 1)
55     return A
56
57
58 def question(n): # 求解线性方程  $H x = b$ 
59     H = H_matrix_func(n)
60     b = [1 for i in range(n)]
61     #  $Hb = plus\_b(H, b)$ 
62     x = Cholesky(H, b)
63     return x
64
65
66 for i in range(1, 11):
67     x = question(i)
68     print(list(x))

```

$n = 1$ 到 $n = 10$ 的各输出结果如下:

1	[1.0]
2	[-2.0000000000000004, 6.000000000000001]
3	[3.0000000000000014, -24.000000000000008, 30.000000000000008]
4	[-3.999999999999389, 59.9999999999265, -179.9999999998172, 139.9999999998786]
5	[4.99999999972516, -119.9999999949219, 629.9999999978285, -1119.9999999967413, 629.9999999984136]
6	[-5.999999999134275, 209.99999999822688, -1679.9999999909319, 5039.999999981689, -6299.999999984089, 2771.9999999950373]
7	[7.00000000946784, -336.0000003682885, 3780.0000034802224, -16800.000013330107, 34650.00002415114, -33264.00002066973, 12012.000006732993]
8	[-7.9999993519750205, 503.999963082772, -7559.999497360088, 46199.997195063115, -138599.9922680939, 216215.9888524956, -168167.99194461384, 51479.99769811924]
9	[8.99992463171293, -719.9946986977008, 13859.90897332794, -110879.34285049817, 450447.5677911754, -1009002.9969426501, 1261254.2185227366, -823676.4894828607, 218789.12870207898]
10	[-9.99647238489706, 989.6931664461617, -23753.43150321029, 240180.04960350535, -1260973.1431035323, 3782989.395580647, -6725420.0812932905, 7000021.515574697, -3937558.2923019733, 923634.2881650471]

两种方法给出的解有差别,且我认为在 n 较大时, Gauss 消元法更为精确。因为解的第一项应为 $(-1)^{n-1}n$, 而 n 较大时, Gauss 消元法比 Cholesky 分解算法得到的解更接近该值。